

LAB-06

Assignment - 1

Develop an ASP.NET Core MVC web application to manage a collection of books. The application must demonstrate the complete operation using the MVC architecture. All book data must be maintained temporarily in a List<Book> in the application. The List should be initialized in a controller.

The system should allow the user to:

View all books

View details of a particular book

Add a new book

Edit an existing book

Delete a book.

Use Book model with suitable properties such as BookId, Title, Author, Category, Price, PublishedYear

Index.cshtml

```
@model IEnumerable<Bookmanagement.Models.Book>
```

```
@{  
    ViewData["Title"] = "Books";  
}
```

```
<h1>Book List</h1>
```

```
<p>  
    <a asp-action="Create" class="btn btn-primary">Add New Book</a>  
</p>
```

```
<table class="table table-bordered">  
    <thead>  
        <tr>
```

```

        <th>Book Id</th>
        <th>Title</th>
        <th>Author</th>
        <th>Price</th>
        <th>Published Year</th>
        <th>Actions</th>
    </tr>
</thead>

<tbody>
    @foreach (var book in Model)
    {
        <tr>
            <td>@book.BookId</td>
            <td>@book.Title</td>
            <td>@book.Author</td>
            <td>@book.Price</td>
            <td>@book.PublishedYear</td>

            <td>
                <a asp-action="Details"
                   asp-route-id="@book.BookId"
                   class="btn btn-info">
                    Details
                </a>

                <a asp-action="Edit"
                   asp-route-id="@book.BookId"
                   class="btn btn-warning">
                    Edit
                </a>

                <a asp-action="Delete"
                   asp-route-id="@book.BookId"
                   class="btn btn-danger">
                    Delete
                </a>
            </td>
        </tr>
    }

```

```
        </td>
      </tr>
    }
  </tbody>
</table>
```

Details.cshtml

@model Bookmanagement.Models.Book

<h1>Book Details</h1>

<div>

<h4>Book</h4>

<hr />

<dl class="row">

<dt class="col-sm-3">Book Id</dt>

<dd class="col-sm-9">@Model.BookId</dd>

<dt class="col-sm-3">Title</dt>

<dd class="col-sm-9">@Model.Title</dd>

<dt class="col-sm-3">Author</dt>

<dd class="col-sm-9">@Model.Author</dd>

<dt class="col-sm-3">Price</dt>

<dd class="col-sm-9">@Model.Price</dd>

<dt class="col-sm-3">Published Year</dt>

<dd class="col-sm-9">@Model.PublishedYear</dd>

</dl>

</div>

<a asp-action="Edit"

asp-route-id="@Model.BookId"

class="btn btn-warning">

Edit


```
<a asp-action="Index" class="btn btn-secondary">  
  Back to List  
</a>
```

Edit.cshtml

@model Bookmanagement.Models.Book

<h1>Edit Book</h1>

```
<form asp-action="Edit" method="post">  
  
  <input type="hidden" asp-for="BookId" />  
  
  <div class="form-group mb-3">  
    <label asp-for="Title"></label>  
    <input asp-for="Title" class="form-control" />  
  </div>  
  
  <div class="form-group mb-3">  
    <label asp-for="Author"></label>  
    <input asp-for="Author" class="form-control" />  
  </div>  
  
  <div class="form-group mb-3">  
    <label asp-for="Price"></label>  
    <input asp-for="Price" class="form-control" />  
  </div>  
  
  <div class="form-group mb-3">  
    <label asp-for="PublishedYear"></label>  
    <input asp-for="PublishedYear" class="form-control" />  
  </div>  
  
  <button type="submit" class="btn btn-primary">
```

Save Changes
</button>

<a asp-action="Index" class="btn btn-secondary">
Cancel

</form>

Delete.cshtml

@model Bookmanagement.Models.Book

<h1>Delete Book</h1>

<h3>Are you sure you want to delete this book?</h3>

<div>
 <p>Title: @Model.Title</p>
 <p>Author: @Model.Author</p>
 <p>Price: @Model.Price</p>
 <p>Published Year: @Model.PublishedYear</p>
</div>

<form asp-action="Delete" method="post">

 <input type="hidden" asp-for="BookId" />

 <button type="submit" class="btn btn-danger">
 Delete
 </button>

 <a asp-action="Index" class="btn btn-secondary">
 Cancel

</form>

[BookController.cs](#)

```
using Bookmanagement.Models;  
using Microsoft.AspNetCore.Mvc;
```

```
namespace Bookmanagement.Controllers  
{  
    public class BookController : Controller  
    {  
        private static List<Book> books = new List<Book>()  
        {  
            new Book{BookId = 1,  
                Title = "The Alchemist",  
                Author = "Paulo Coelho",  
                Price = 299,  
                PublishedYear = 1988},  
            new Book  
            {  
                BookId = 2,  
                Title = "Clean Code",  
                Author = "Robert C. Martin",  
                Price = 599,  
                PublishedYear = 2008  
            },  
            new Book  
            {  
                BookId = 3,  
                Title = "Harry Potter",  
                Author = "J.K. Rowling",  
                Price = 499,  
                PublishedYear = 1997  
            }  
        };  
        public IActionResult Index()  
        {  
            return View(books);  
        }  
        public IActionResult Details(int id)
```

```

{
    var book = books.FirstOrDefault(b => b.BookId == id);

    if (book == null)
    {
        return NotFound();
    }

    return View(book);
}
public IActionResult Create() {
    return View();
}
[HttpPost]
public IActionResult Create(Book book)
{
    if (ModelState.IsValid)
    {
        book.BookId = books.Count > 0
            ? books.Max(b => b.BookId) + 1
            : 1;

        books.Add(book);

        return RedirectToAction(nameof(Index));
    }

    return View(book);
}
public IActionResult Edit(int id)
{
    var book = books.FirstOrDefault(b => b.BookId == id);

    if (book == null)
    {
        return NotFound();
    }
}

```

```

        return View(book);
    }
    [HttpPost]
    public IActionResult Edit(int id, Book book)
    {
        if (id != book.BookId)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
            var existingBook = books.FirstOrDefault(b => b.BookId == id);

            if (existingBook == null)
            {
                return NotFound();
            }

            existingBook.Title = book.Title;
            existingBook.Author = book.Author;
            existingBook.Price = book.Price;
            existingBook.PublishedYear = book.PublishedYear;

            return RedirectToAction(nameof(Index));
        }

        return View(book);
    }
    public IActionResult Delete(int id)
    {
        var book = books.FirstOrDefault(b => b.BookId == id);

        if (book == null)
        {
            return NotFound();
        }
    }

```

```

    }

    return View(book);
}
[HttpPost]
[ActionName("Delete")]
public IActionResult DeleteConfirmed(int id)
{
    var book = books.FirstOrDefault(b => b.BookId == id);

    if (book == null)
    {
        return NotFound();
    }

    books.Remove(book);

    return RedirectToAction(nameof(Index));
}
}
}

```

[Book.cs](#)

```

namespace Bookmanagement.Models
{
    public class Book
    {
        public int BookId { get; set; }
        public string Title { get; set; }
        public string Author { get; set; }
        public int Price { get; set; }
        public int PublishedYear { get; set; }
    }
}

```

Books - Bookmanagement

https://localhost:7143

☆ ☆ ⋮ Chat

BookmanagementHomePrivacy

Book List

Add New Book

Book Id	Title	Author	Price	Published Year	Actions
1	The Alchemist	Paulo Coelho	299	1988	DetailsEditDelete
2	Clean Code	Robert C. Martin	599	2008	DetailsEditDelete

© 2026 - Bookmanagement - [Privacy](#)

Search

ENG IN12:0717-08-2026

Assignment - 2

Modify the application developed in Assignment-1 to use Repository pattern. Define IBookRepository interface and BookRepository class which implements all CRUD operations.

[program.cs](#)

```
using Bookmanagement.Repositories;
using BookManagement.Repositories;
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddScoped<IBookRepository, BookRepository>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for
    production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
```

```
pattern: "{controller=Book}/{action=Index}/{id?}")
.WithStaticAssets();
```

```
app.Run();
```

[IBookRepository.cs](#)

```
using Bookmanagement.Models;
```

```
namespace Bookmanagement.Repositories
```

```
{
    public interface IBookRepository
    {
        List<Book> GetAll();

        Book GetById(int id);

        void Add(Book book);

        void Update(Book book);

        void Delete(int id);
    }
}
```

[BookRepository.cs](#)

```
using Bookmanagement.Models;
```

```
using Bookmanagement.Repositories;
```

```
namespace BookManagement.Repositories
```

```
{
    public class BookRepository : IBookRepository
    {
        private static List<Book> books = new List<Book>()
```

```

{
    new Book
    {
        BookId = 1,
        Title = "The Alchemist",
        Author = "Paulo Coelho",

        Price = 299,
        PublishedYear = 1988
    },

    new Book
    {
        BookId = 2,
        Title = "Clean Code",
        Author = "Robert C. Martin",

        Price = 599,
        PublishedYear = 2008
    },

    new Book
    {
        BookId = 3,
        Title = "Harry Potter",
        Author = "J.K. Rowling",

        Price = 499,
        PublishedYear = 1997
    }
};

public List<Book> GetAll()
{
    return books;
}

```

```
public Book GetById(int id)
{
    return books.FirstOrDefault(b => b.BookId == id);
}

public void Add(Book book)
{
    book.BookId = books.Count > 0
        ? books.Max(b => b.BookId) + 1
        : 1;

    books.Add(book);
}

public void Update(Book book)
{
    var existingBook = books.FirstOrDefault(
        b => b.BookId == book.BookId
    );

    if (existingBook != null)
    {
        existingBook.Title = book.Title;
        existingBook.Author = book.Author;
        existingBook.Price = book.Price;
        existingBook.PublishedYear = book.PublishedYear;
    }
}

public void Delete(int id)
{
    var book = books.FirstOrDefault(
        b => b.BookId == id
    );

    if (book != null)
    {

```

```

        books.Remove(book);
    }
}
}

```

[BookController.cs](#)

```

using Bookmanagement.Models;
using Bookmanagement.Repositories;
using Microsoft.AspNetCore.Mvc;

namespace BookManagement.Controllers
{
    public class BookController : Controller
    {
        private readonly IBookRepository repository;

        public BookController(IBookRepository repository)
        {
            this.repository = repository;
        }

        // GET: Book
        public IActionResult Index()
        {
            var books = repository.GetAll();

            return View(books);
        }

        // GET: Book/Details/1
        public IActionResult Details(int id)
        {
            var book = repository.GetByld(id);

            if (book == null)
            {

```

```

        return NotFound();
    }

    return View(book);
}

// GET: Book/Create
public IActionResult Create()
{
    return View();
}

// POST: Book/Create
[HttpPost]
public IActionResult Create(Book book)
{
    if (ModelState.IsValid)
    {
        repository.Add(book);

        return RedirectToAction(nameof(Index));
    }

    return View(book);
}

// GET: Book/Edit/1
public IActionResult Edit(int id)
{
    var book = repository.GetByld(id);

    if (book == null)
    {
        return NotFound();
    }

    return View(book);
}

```

```
}
```

```
// POST: Book/Edit/1
```

```
[HttpPost]
```

```
public IActionResult Edit(int id, Book book)
```

```
{
```

```
    if (id != book.BookId)
```

```
    {
```

```
        return NotFound();
```

```
    }
```

```
    if (ModelState.IsValid)
```

```
    {
```

```
        repository.Update(book);
```

```
        return RedirectToAction(nameof(Index));
```

```
    }
```

```
    return View(book);
```

```
}
```

```
// GET: Book/Delete/1
```

```
public IActionResult Delete(int id)
```

```
{
```

```
    var book = repository.GetByld(id);
```

```
    if (book == null)
```

```
    {
```

```
        return NotFound();
```

```
    }
```

```
    return View(book);
```

```
}
```

```
// POST: Book/Delete/1
```

```
[HttpPost, ActionName("Delete")]
```

```

public IActionResult DeleteConfirmed(int id)
{
    var book = repository.GetByld(id);

    if (book == null)
    {
        return NotFound();
    }

    repository.Delete(id);

    return RedirectToAction(nameof(Index));
}
}
}

```

